

附录 4: Verilog 数字系统设计基础

一、 Verilog HDL（硬件描述语言）概述

硬件描述语言HDL（Hardware Description Language）是一种用形式化方法来描述数字电路和数字逻辑系统的语言。数字逻辑电路设计者可利用这种语言来描述自己的设计思想，然后利用EDA工具进行仿真，再自动综合到门级电路，最后用ASIC或FPGA实现其功能。举个例子，在传统的设计方法中，对2输入的与门，我们可能需到标准器件库中调出74系列器件，但在硬件描述语言中，“&”就是一个与门的形式描述，“C = A & B”就是一个2输入与门的描述。而“and”就是一个与门器件。硬件描述语言发展至今已有二十多年历史，当今业界的标准中（IEEE标准）主要有VHDL和Verilog HDL这两种硬件描述语言。

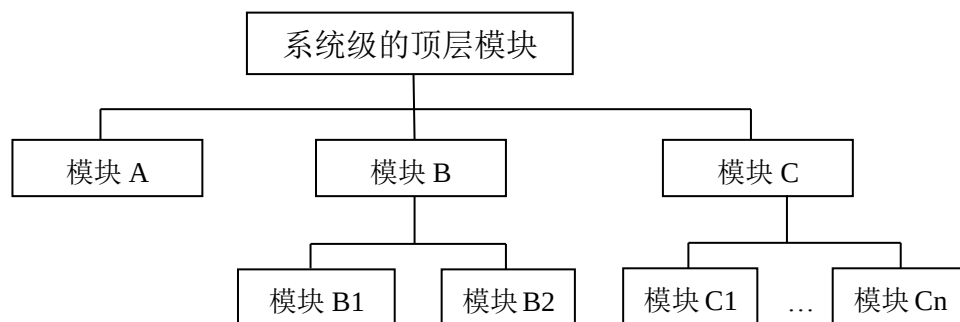
Verilog HDL语法结构上的主要特点：

- 1.形式化的表示电路的行为和功能；
- 2.借用C语言的结构和语句；
- 3.可在多个层次上对所设计的系统加以描述，语言对设计规模不做限制；
- 4.具有混合建模能力，一个设计中的各子模块可用不同级别的抽象模型来描述；
- 5.基本逻辑门、开关级结构模型均内置在语言中，可直接调用；
- 6.易创建用户定义原语。

二、 采用 Verilog HDL 的设计流程

1.Top_Down（自顶向下）的设计方法

当前的集成电路的设计有多种设计方法，但一般地采用自顶向下的设计方法。因为随着技术的发展，一个芯片上往往集成了几十万到几百万个器件，传统的自底向上的设计方法已不太现实。因此，一个设计往往从系统级设计开始，把系统划分成几个大的基本的功能模块，每个功能模块再按一定的规则分成下一个层次的基本单元，如此一直划分下去。自顶向下的设计方法可用下面的树状结构表示：



通过自顶向下的设计方法，可实现设计的结构化，使一个复杂的系统设计可由多个设计者分工合作，还可以实现层次化的管理。

2.Verilog HDL建模

模块（module）是Verilog的基本描述单位，用于描述某个设计的功能或结构及与其他模块通信的外部端口。

模块在概念上可等同一个器件就如我们调用通用器件（与门、三态门等）或通用宏单元（计数器、ALU、CPU）等，因此，一个模块可在另一个模块中调用。

一个电路设计可由多个模块组合而成，因此一个模块的设计只是一个系统设计中的某个层次设计，模块设计可采用多种建模方式。

一个模块的基本语法如下：

```

module module_name (port1, port2, .....);
// Declarations 以下是声明区:
input, output, inout,
reg, wire, parameter,
function, task, ...
//Statements 以下是语句区:
Initial statement
Always statement
Module instantiation
Gate instantiation
Continuous assignment
endmodule
  
```

模块的结构需按上面的顺序进行，声明区用来对信号方向、信号数据类型、函数、任务、参数等进行描述。语句区用来对功能进行描述如：器件调用（Module instantiation）等。

例[1] 三态驱动器

```

module mytri (din, d_en, d_out);
input din;
input d_en;
  
```

```

output d_out;
// -- Enter your statements here -- //
assign d_out = d_en ? din : 'bz;
endmodule
module trist (din, d_en, d_out);
input din;
input d_en;
output d_out;
// -- statements here -- //
mytri u_mytri(din,d_en,d_out);
endmodule

```

例[1]描述了一个三态驱动器。其中三态驱动门在模块 `mytri` 中描述，而在模块 `trist` 中调用了模块 `mytri`。模块 `mytri` 对 `trist` 而言相当于一个已存在的器件，在 `trist` 模块中对该器件进行实例化，实例化名 `u_mytri`。通过例[1]可以看出：

(1) 模块内容是嵌在 `module` 和 `endmodule` 两个语句之间。每个模块实现特定的功能，模块可进行层次的嵌套，因此可以将大型的数字电路设计分割成大小不一的小模块来实现特定的功能，最后通过由顶层模块调用子模块来实现整体功能，这就是 **Top-Down** 的设计思想，如例[1]。

(2) 模块包括接口描述部分和逻辑功能描述部分。这可以把模块与器件相类比。

模块的端口定义部分如：`module mytri (din, d_en, d_out);` 其中 `module` 是模块的保留字，`mytri` 是模块的名字，相当于器件名。（）内是该模块的端口声明，定义了该模块的管脚名，是该模块与其他模块通讯的外部接口，相当于器件的引脚。

模块的内容包括 I/O 说明，内部信号、调用模块等的声明语句和功能定义语句。

I/O 说明语句如：`input din; input d_en; output d_out; input [2:0] a;` 其中的 `input`、`output`、是保留字，定义了管脚信号的流向，`[n:0]` 表示该信号的位宽（总线或单根信号线）。

逻辑功能描述部分如：`assign d_out = d_en ? din : 'bz;`

`mytri u_mytri(din,d_en,d_out);`

功能描述用来产生各种逻辑，还可用来实例化一个器件，该器件可以是厂家的器件库也可以是我们自己用 HDL 设计的模块（相当于在原理图输入时调用一个库元件）。

(3) 对每个模块都要进行端口定义，并说明输入、输出口，然后对模块的

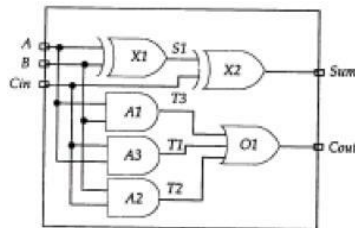
功能进行逻辑描述。对测试模块，可以没有输入输出口。

(4) 除endmodule 语句外，每个语句后面需有分号表示该语句结束。

在HDL的建模中，主要有三种建模方式，分别是结构化描述方式、数据流描述方式和行为描述方式。

(1) 结构化描述的建模方式就是通过对电路结构的描述来建模，即通过对器件的调用（HDL概念称为例化），并使用线网来连接各器件的描述方式。结构化的描述方式反映了一个设计的层次结构。

例[2]：一位全加器（结构图如下）



代码：

```
module FA_struct (A, B, Cin, Sum, Count);
input A;
input B;
input Cin;
output Sum;
output Count;
wire S1, T1, T2, T3;
// -- statements -- //
xor x1 (S1, A, B);
xor x2 (Sum, S1, Cin);
and A1 (T3, A, B );
and A2 (T2, B, Cin);
and A3 (T1, A, Cin);
or O1 (Cout, T1, T2, T3 );
endmodule
```

该实例显示了一个全加器由两个异或门、三个与门、一个或门构成。S1、T1、T2、T3则是门与门之间的连线。代码显示了用纯结构的建模方式，其中xor、and、or是Verilog HDL内置的门器件。以xor x1 (S1, A, B)该例化语句为例：xor表明调用一个内置的异或门，器件名称xor，代码实例化名x1（类似原理图输入方式）。括号内的S1, A, B表明该器件管脚的实际连接线（信号）的名称，其中A、B是输入，S1是输出。

(2) 数据流描述的建模方式就是通过对数据流在设计中的具体行为的描述

的来建模。最基本的机制就是用连续赋值语句。在连续赋值语句中，某个值被赋给某个线网变量（信号），语法如下：

```
assign [delay] net_name = expression;
```

如：assign #2 A = B;

在数据流描述方式中，还必须借助于HDL提供的一些运算符，如按位逻辑运算符：逻辑与（&），逻辑或（|）等。以一位的全加器为例，可用如下的建模方式：

```
`timescale 1ns/100ps
module FA_flow(A,B,Cin,Sum,Count)
input A,B,Cin;
output Sum, Count;
wire S1,T1,T2,T3;
assign # 2 S1 = A ^ B;
assign # 2 Sum = S1 ^ Cin;
assign #2 T3 = A & B;
assign #2 T1 = A & Cin;
assign #2 T2 = B & Cin ;
endmodule
```

注意在各assign语句之间，是并行执行的，即各语句的执行与语句之间的顺序无关。如上，当A有个变化时，S1、T3、T1 将同时变化，S1的变化又会造成Sum的变化。

（3）行为描述的建模方式是指采用对信号行为级的描述（不是结构级的描述）的方法来建模。在表示方面，类似数据流的建模方式，但一般是把用initial 块语句或always块语句描述的归为行为建模方式。行为建模方式通常需要借助一些行为级的运算符如加法运算符（+），减法运算符（-）等。以一位全加器为例的行为建模如下：

```
module FA_behav1(A, B, Cin, Sum, Cout );
input A,B,Cin;
output Sum,Cout;
reg Sum, Cout;
reg T1,T2,T3;
always@ ( A or B or Cin )
begin
Sum = (A ^ B) ^ Cin ;
T1 = A & Cin;
T2 = B & Cin ;
T3 = A & B;
Cout = (T1| T2) | T3;
```

```
end  
endmodule
```

需要注意：

只有寄存器类型的信号才可以在always和initial 语句中进行赋值，类型定义通过reg语句实现。

always语句是从0时刻开始一直重复执行，由敏感表（always 语句括号内的变量）中的变量触发。

在begin和end之间的语句是顺序执行，属于串行语句。

一位全加器的另一种行为建模如下：

```
module FA_behav2(A, B, Cin, Sum, Cout );  
input A,B,Cin;  
output Sum,Cout;  
reg Sum, Cout;  
always@ ( A or B or Cin )  
begin  
{Count , Sum} = A + B + Cin ;  
end  
endmodule
```

在上例中，采用更加高级（更趋于行为级）描述方式，即直接采用“+”来描述加法。{Count, Sum}表示对位数的扩展，因为两个1bit 相加，和有两位，低位放在Sum变量中，进位放在Count中。

（4）混合设计描述方式在实际的设计中，往往是多种设计模型的混合。一般地，对顶层设计，采用结构描述方式，对底层模块，可采用数据流、行为级或两者的结合。

三、 Verilog HDL 基本语法

1.基本语法定义

Verilog HDL 源代码是由大量的基本语法元素构成，其中包括空白部分（White space）、注释（Comment）、运算符（Operator）、数值（Number）、字符串（String）、标识符（Identifier）和关键字（Keyword）。

（1）注释

在代码中添加注释行可以提高代码的可读性和可维护性。Verilog HDL 中注释行的定义与 C 语言完全一致，分为两类：第一类是单行注释，以“//”开始到本行行末结束，不允许续行。第二类是多行注释，以“/*”开始，以“*/”结束。可以

跨越多行，但是中间不允许嵌套。

(2) 运算符

Verilog HDL 中运算符可以分以下几类：

运算符分类	所含运算符
算术运算符	+, -, *, /, %
关系运算符	<, >, <=, >=
相等与全等运算符	=, !=, ==, !==
逻辑运算符	!, &&,
位运算符	~, &, , ^, ^~或~^
缩位运算符	&, ~&, , ~ , ^, ^~或~^
逻辑移位运算符	<<, >>
位拼接运算符	{ }
条件运算符	?:

(3) 数值

Verilog HDL 的数值集合由以下四个基本的值组成：

0—— 代表逻辑 0 或假状态；

1——代表逻辑 1 或真状态；

X（或 x）——代表逻辑不定态；

Z（或 z）——代表高阻态。

常数按照其数值类型可以划分为整数和实数两种。Verilog HDL 的整数可以是十进制、十六进制、八进制或二进制。

整数定义的格式为：

<位宽>'<基数><数值>

位宽：描述常量所含二进制数的位数，用十进制整数表示，是可选项，如果没有这一项，可以从常量的值推断出。

基数：可选项，可以是b(B), o(O), d(D), h(H)分别表示二进制、八进制、十进制和十六进制。基数缺省默认为十进制数。

数值：是由基数所决定的表示常量真实值的一串ASCII码。如果基数定义为b或B，数值可以是0, 1, x(X), z(Z)。对于基数是d或D的情况，数值符可以是0到9的任何十进制数，但不可以是X或Z。例如

5 'O37 5位八进制数（二进制 11111 ）

4 'D2 4位十进制数（二进制0011）

4 'B1x_01 4位二进制数

7'Hx 7位x(扩展的x), 即xxxxxxx

(4) 字符串

字符串指同一行内写在双引号之间的字符序列串, 在表达式和赋值语句中字符串用作操作数, 而且要转换成无符号整型常量, 用若干个 8 位二进制 ASCII 码的形式表示, 其中每 8 位二进制 ASCII 码代表一个字符。例如, 字符串“ab”等价于 16'h5758。

(5) 标识符

用于定义模块名、端口名、信号名等。 Verilog HDL 中的标识符可以是任意一组字母、数字、\$符号和_(下划线)符号的组合, 但标识符的第一个字符必须是字母或者下划线。另外, 标识符是区分大小写的。例如

Count

COUNT //与Count 不同。

(6) 关键字

Verilog HDL 定义了一系列保留字, 叫做关键词, 注意只有小写的关键词才是保留字。Verilog HDL中约有98个关键词, 其中常用的有:

always endmodule reg and assign begin for
case or function output parameter wait if else
input while end

2.常用数据类型

Verilog HDL 主要包括两种数据类型, 线网类型(net type) 和寄存器类型(reg type)。

(1) 线网类型

线网类型主要有wire和tri两种。线网类型用于对结构化器件之间的物理连线的建模。如器件的管脚, 内部器件如与门的输出等。由于线网类型代表的是物理连接线, 因此它不存贮逻辑值。必须由器件所驱动。通常由assign进行赋值。如
assign A = B ^ C;

当一个wire类型的信号没有被驱动时, 缺省值为Z (高阻)。信号没有定义数据类型时, 缺省为wire类型。tri主要用于定义三态的线网。

(2) 寄存器类型

reg是最常用的寄存器类型，寄存器类型通常用于对存储单元的描述，如D型触发器、ROM等。存储器类型的信号当在某种触发机制下分配了一个值，在分配下一个值之时保留原值。但必须注意的是，reg类型的变量，不一定是存储单元，如在always语句中进行描述的必须用reg类型的变量。

reg 类型定义语法如下：

reg [msb: lsb] reg1, reg2, ... reg N;

msb和lsb定义了范围，并且均为常数值表达式。范围定义是可选的；如果没有定义范围，缺省值为1位寄存器。例如：

reg [3:0] Sat; // Sat 为4位寄存器。

reg Cnt; //1 位寄存器Cnt。

3.条件语句

(1) if语句的语法如下：

```
if(condition_1)
    procedural_statement_1
{else if(condition_2)
    procedural_statement_2}
{else
    procedural_statement_3}
```

如果对condition_1求值的结果为个非零值，那么procedural_statement_1被执行，如果condition_1的值为0、x或z，那么procedural_statement_1不执行。如果存在一个else分支，那么这个分支被执行。

(2) case语句是一个多路条件分支形式，其语法如下：

```
case(case_expr)
case_item_expr{ ,case_item_expr} :procedural_statement
...
...
[default:procedural_statement]
endcase
```

case语句首先对条件表达式case_expr求值，然后依次对各分支项求值并进行比较，第一个与条件表达式值相匹配的分支中的语句被执行。可以在1个分支中定义多个分支项；这些值不需要互斥。缺省分支覆盖所有没有被分支表达式覆盖的其他分支。

4.实例化语句

一个模块能够在另外一个模块中被引用，这样就建立了描述的层次。模块实例化语句形式如下：

```
module_name instance_name(port_associations);
```

信号端口可以通过位置或名称关联；但是关联方式不能够混合使用。端口关联形式如下：

```
port_expr //通过位置。
```

```
.PortName (port_expr) //通过名称。
```

例[3]:

```
....
module and (C, A, B);
input A, B;
output C;
...
and A1 (T3, A, B); //实例化时采用位置关联，T3对应输出端口C，A
对应A，B对应B。
and A2 ( //实例化时采用名字关联，.C是and 器件的端口，其与信号
T3相连
.C (T3),
.A (A),
.B (B)
);
```

5.顺序语句块(begin...end)

顺序语句块中的语句按顺序方式执行。每条语句中的时延值与其前面的语句执行的模拟时间相关。一旦顺序语句块执行结束，跟随顺序语句块过程的下一条语句继续执行。顺序语句块的语法如下：

```
begin
[ :block_id{declarations} ]
procedural_statement ( s )
end
```

6.过程赋值语句

Verilog HDL 中提供两种过程赋值语句，分别是initial和 always 语句，用这两种语句来实现行为的建模。这两种语句之间的执行是并行的，即语句的执行与位置顺序无关。这两种语句通常与语句块（beginend）相结合，则语句块中的执行是按顺序执行的。

(1) initial 语句

initial语句只执行一次，即在设计被开始模拟执行时开始（0时刻）。通常只

用在对设计进行仿真的测试文件中，用于对一些信号进行初始化和产生特定的信号波形。语法如下：

```
initial
[timing_control] procedural_statement
```

(2) always语句

always语句与initial语句相反，是被重复执行，执行机制是通过对一个称为敏感变量表的事件驱动来实现的。always语句可实现组合逻辑或时序逻辑的建模。

例[4]:

```
initial
Clk = 0 ;
always
#5 Clk = ~Clk;
```

因为always语句是重复执行的，因此Clk是初始值为0的，周期为10的方波。

例[5] 2 选一的分配器

```
always @( sel , a , b )
C = sel ? a : b;
```

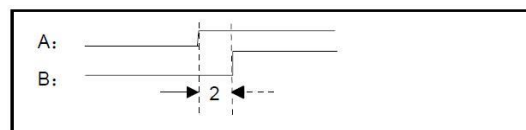
这里的sel, a, b同样称为敏感变量，当三者之一有变化时，always被执行，当sel为1，C被赋值为a，否则为b。

7.仿真时间尺度

信号在电路传输会有传播延时等，如线延时、器件延时。时延就是对延时特性的HDL描述。举例如下：

```
assign # 2 B = A;
```

表示B信号在2个时间单位后得到A信号的值。如下图：



在Verilog HDL中，所有时延都必须根据时间单位进行定义，定义方式为在文件头添加如下语句：

```
`timescale 1ns /100ps
```

其中'timescale是Verilog HDL提供的预编译处理命令，1ns表示时间单位是1ns,100ps表示时间精度是100ps。根据该命令，编译工具才可以认知#2为2ns。在Verilog HDL的IEEE标准中没有规定时间单位的缺省值，由各模拟工具确定。因此在写代码时必须确定仿真时间尺度。